by replaying the WAL. The WAL will be deleted when a new snapshot is generated.

To take a snapshot for a vector index, we need to record both the in-memory and on-disk data structures. For in-memory index data, we create snapshots for centroid index, version maps in *Updater*, and block mapping and block pool in *Block Controller*, and flush the snapshots to disk. Snapshots are relatively cheap because these data structures take only 40GB for billion-level dataset, which costs 2~3 seconds for a full flush on a PCI-e based NVM SSD. For disk data, thanks to our block-level copy-on-write mechanism, we can collect all the released blocks during two snapshots into a *pre-release* buffer, which will be added to the *Free Block Pool* after the next snapshot is recorded. Thus all the data blocks modified in the interim can be rolled back to be consistent with the previous snapshot. This solution saves a large amount of disk space since we only delay the space release for old blocks during two snapshots.

5 Evaluation

In this section, we conduct experiments to answer the following questions:

- How does SPFresh compare with state-of-the-art baselines in terms of performances, search accuracy and resource usage? (§5.2)
- What is the maximum performance of SPFresh? (§5.3)
- Can SPFresh solve the data shifting problem illustrated in Figure 2? (§5.4)
- How to properly configure SPFresh? (§5.5)

5.1 Experimental Setup

Platform: All experiments run on an Azure lsv3 [41] VM instance, which is a storage-optimized virtual machine with locally attached high-performance NVMe storage. In particular, we configured the VM with 16 vCPUs from a hyper-threaded Intel Xeon Platinum 8370C (Ice Lake) processor and 128GB memory for our experiments.

Datasets: We use two widely-used vector datasets to evaluate SPFresh:

- SIFT1B [40] is a classical image vector dataset for evaluating the performance of ANNS algorithms that support large-scale vector search. It contains one billion of 128-dimensional byte vectors as the base set and 10,000 query vectors as the test set.
- SPACEV1B [1] is a dataset derived from production data from commercial search engines. It represents a different form of vector encoding: deep natural language encoding. It contains one billion of 100-dimensional byte vectors as a base set and 29,316 query vectors as the test set.

Baselines: We compare SPFresh with two baselines:

- *DiskANN* is the state-of-the-art disk-based fresh ANNS system [53]. It is based on a graph ANNS index and uses an out-of-place update solution. We configure DiskANN with the same settings as in their paper [53]. For update configurations, DiskANN baseline processes *streamingMerge*, a lightweight global graph rebuild, for every new 30M vectors, where graph degree R equals 64 and insert candidate list equals 75. For search configurations, DiskANN baseline uses the default setting with beamwidth equal to 2 and search candidate list L equal to 40 for recall10@10.
- *SPANN+*, a modified version of SPANN [67] which appends updates locally to a posting *without splitting and reassigning*. This is an *append-only* version of SPFresh without the *Local Rebuilder* module.

Workloads: Three workloads are used in the experiments.

- *Workload A* simulates a realistic vector update scenario with 100 million scale of SPACEV vectors. The reason to reduce the scale from 1 billion to 100 million is that DiskANN requires several TBs of DRAM to run 1-billion scale fresh updates (shown in Table 1), which exceeds our machine's capacity. In particular, workload A simulates 1% update daily over 100 days. To generate updates realistically, we extract two disjoint SPACEV 100M datasets from SPACEV1B, where one is used as the base ANNS index data-set and the other as the update candidate pool. Each daily update epoch deletes 1% of vectors randomly from the base ANNS index, and inserts 1% of vectors randomly selected from the update data pool to the base index.
- *Workload B* has the same data scale and sampling method as Workload A but with a 100 million scale of the SIFT vector dataset.
- *Workload C* scales up our experiment data-set to be *billion-scale* using both the SIFT dataset and the SPACEV dataset. This workload aims at stress testing SPFresh, also with a 1% daily update rate.

Metrics: SPFresh is designed for online ANNS streaming scenarios. Thus, our evaluation focuses on the following four categories of metrics.

- Search Performance: We measure tail (P90, P95, P99, and P99.9) latency and query per second (QPS) throughput. In particular, we have a hard cut of 10ms for SPFresh and all baselines, where the system finishes the result immediately and returns the current search results.
- Search Accuracy: We use the percentage of ground truths recalled by SPFresh system to measure accuracy.
- Update Performance: insertion and deletion throughputs.
- Resource Usages: the memory and CPU consumption.

5.2 Real-World Update Simulation

In this experiment, we compare all the metrics of SPFresh with all the baselines on the real-world situation. We use Workload A and B (see §5.1) to simulate 100 days of real-world updates, and we show that SPFresh outperforms baselines in all evaluation metrics.

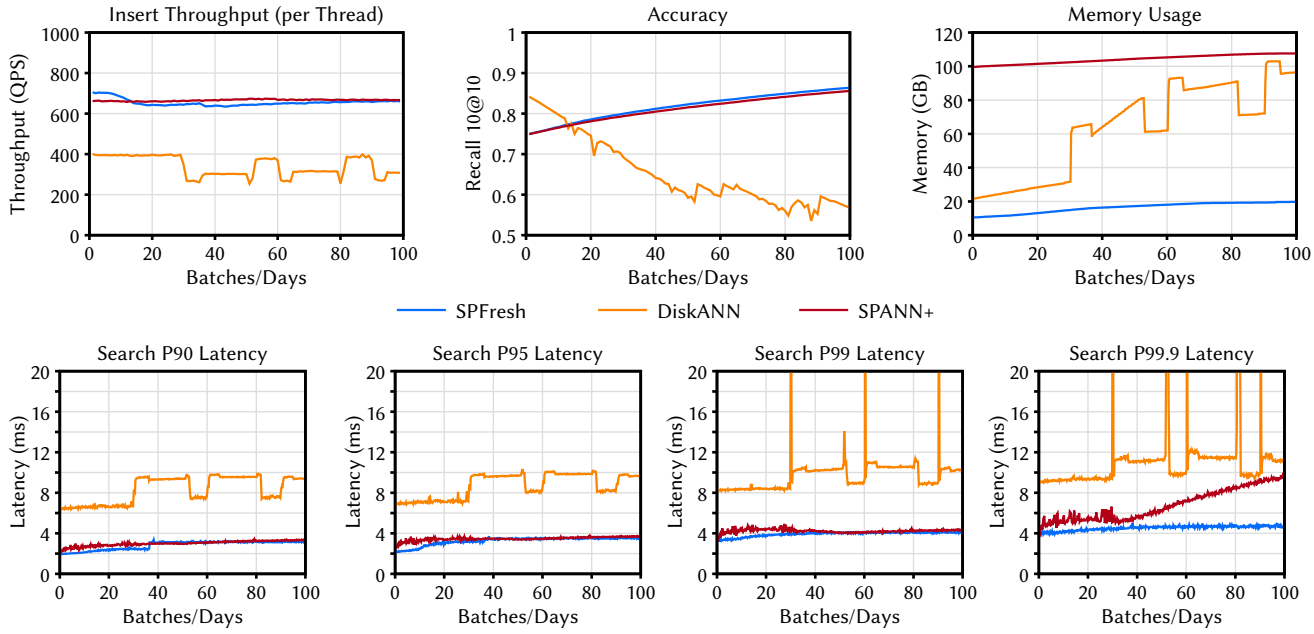

Figure 7. SPFresh overall performance (SPACEV: data distribution shifts over time).

	DiskANN	SPANN+	SPFresh
Insert	3	1	1
delete	1	1	1
Search	2	2	2
Background	10	2	2
Total	16	6	6

Table 2. Threads allocation for overall performance.

5.2.1 System Setup. Table 2 lists the thread allocation for each system. Specifically, we allocate threads to each system's sub-components to meet the processing requirements of handling update throughput of 600~1200 QPS. The setting of update QPS is based on Alibaba's daily update speed (100 million each day) [69]. Each system only needs one thread to serve delete requests because deletion uses tombstones to record the deletions, which is lightweight. For DiskANN, due to its high insert latency, we set its number of insert threads to 3 and the number of background merge threads to 10, because it is the minimum to keep up with the update and garbage collection process. Further increasing the number will impact the query performance in the foreground negatively. The two remaining threads are used for foreground search for DiskANN.

To be comparable with DiskANN, SPFresh and SPANN+ also set 2 search threads. Both SPANN+ and SPFresh only need one insert thread to serve 600+ QPS insert throughput and two background threads for SPDK I/O and garbage collection (or local rebuilder).

5.2.2 Experiment Results. Figure 7 records a daily time series of the search tail latency, insert throughput per thread, search accuracy, and memory usage of Workload A. We can see that SPFresh achieves the best and the most stable performance on all the metrics during the 100 days.

Low and Stable Search Tail Latency: Figure 7 shows that SPFresh achieves low and stable tail latency in all percentile measurements. Since the overall tail latency trends are similar, we focus our discussion on the most stringent tail latency measurement, P99.9.

Experiments indicate that LIRE is able to keep posting distribution uniform because SPFresh has a stable low search P99.9 latency around 4ms. In comparision, other systems' P99.9 latency is both worse and less stable than SPFresh.

DiskANN's P99.9 latency fluctuates significantly with a dramatic increase to more than 20ms during global rebuilds because a search thread could be blocked by a global rebuild even with 10ms hard latency cut. The search P99.9 latency of SPANN+ increases significantly from 4ms to more than 10ms because its posting keeps growing, inducing data skews and the increase of I/O and computation cost.

Overall, SPFresh maintains 2.41x lower P99.9 latency to DiskANN on average and expands its latency advantage to SPANN+. The low and stable search tail latency can be attributed to the LIRE protocol. During the experiment, we found that only 0.4% insertion will cause rebalancing. Among them, the average split number is 2, and the maximum split

number is 160, with a cascading length of 3. The merge frequency is only 0.1% of the update (insertion and deletion). On average, each time 5094 vectors are evaluated, and only 79 are actually reassigned.

High Search Accuracy: SPFresh achieves a higher search accuracy compared to baselines. Both SPANN+ and SPFresh would not violate the NPA of a cluster-based index. Therefore, the accuracy of SPANN+ and SPFresh grows gradually since newly inserted vectors are all assigned to a subset of postings due to the data shifting. Therefore, queries to these new vectors can easily hit since search and insertion follow the same search path to get the nearest postings.

Although SPANN+ search accuracy increases in a similar trend like SPFresh, the gap in accuracy increases over time. The increasing gap between these two systems is because the index quality of SPANN+ degrades as partition distributions skew over time.

DiskANN proposes an algorithm to reduce the overhead of global rebuild by eliminating outdated edges from all vertices and populating edges for a new vertex using the neighborhood of its deleted neighbors. This method aims to reduce the decline in accuracy due to a decreased number of edges caused by vector deletions without reconstructing its graph-based index completely. However, experiments show that such a method cannot prevent DiskANN's search accuracy from decreasing over time.

High and Stable Update Performance: SPFresh achieves 1.5ms average insert latency and stable tail latency. On the other hand, DiskANN suffers from the heavy computation caused by in-memory graph traversal, and thus results in higher latency and lower throughput. Compared to SPANN+, we can see that SPFresh's lightweight *Local Rebuilder* will not affect the foreground insert performance.

Low Resource Utilization: For resource usage, SPFresh achieves as low as 5.30X lower memory usage than baselines during the whole update process. DiskANN occupies an extra 60G memories for background streamingMerge and 15GB for the second in-memory index for the update. SPANN+ needs much larger block-mapping entries to allow a larger posting length. SPFresh keeps the memory under 20GB, which only grows slightly over time because new metadata are created for each new posting triggered by splits. SPFresh also maintains a reasonable disk size. In the index, we find that 86% of the total vectors have more than one replica, and on average, one vector has 5.47 replicas, which is similar to the index built statically.

We also ran the same experiment on Workload B and reached a similar conclusion with DiskANN. Note that SPANN+ achieves similar performance with SPFresh on the SIFT dataset, which is almost uniformly distributed. This is expected because background garbage collection should be able to prune stale vectors on SPANN+ without splits on uniform data-set. Consequently, SPANN+ achieves a similar index quality as that of SPFresh because its posting distribution does not shift much.

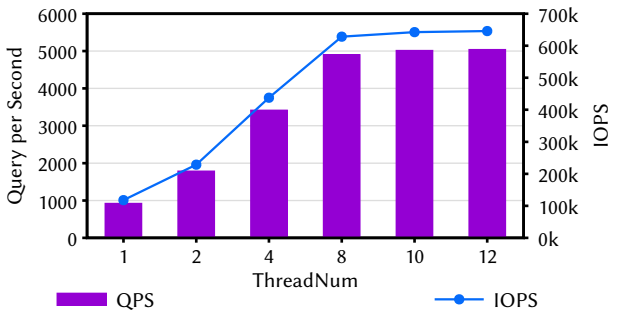

Figure 8. Search throughput/disk IOPS vs # of SPFresh search threads on Azure lsv instance [41].

	#threads		#threads
Delete/Re-insert	4	Search	8
Background	3	Total	15

Table 3. Thread allocation for SPFresh in billion-scale tests.

5.3 Billion-Scale Stress Test

We scale up vector data size to billion-level and configure the system to show the best performance SPFresh achieves with the given resource. We use Workload C (see §5.1) to simulate a 20-day real-world update scenario. We demonstrate that SPFresh has fully saturated SSD's bandwidth and performed well with stable resource utilization.

5.3.1 System Setup. Table 3 lists thread allocation for SPFresh's stress tests. To fully achieve the IOPS of SSD, our setting maximizes search throughput while supporting maximum update throughput.

Azure lsv3 has a max *guaranteed* NVMe IOPS of 400K [41]. We first run an experiment to find out the max search throughput lsv3 can handle. As Figure 8 shows, the IOPS and search throughput almost reach the peak at 8 search threads on a single SSD disk.

When search thread count is set to 8 for max search throughput, a fore-ground thread count of 4 saturates the update throughput. Therefore we set the thread counts as in Table 3 for this experiment. Search is the most important part of ANNS service, so the stress test we will maximize our search throughput while support the maximum update throughput. Since Azure document[41] note that the max NVMe disk throughput is 400K and can go higher but not be guaranteed to keep the IOPS higher than 400K, we make a simple test on lsv3 NVMe SSD by running SPFresh Search On Azure lsv3 to find out that the MAX performance of lsv3 NVMe SSD. and we can see on figure 8 that when we set

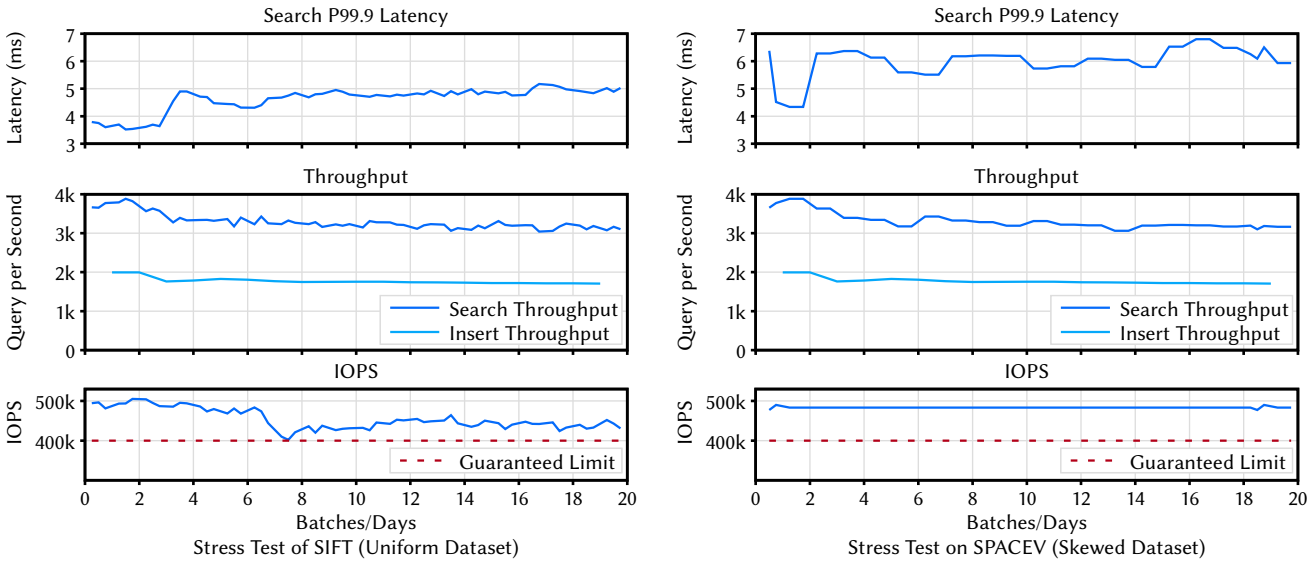

Figure 9. Billion-scale stress test on uniform and skew datasets. SPFresh saturates the I/O with a stable P99.9 latency while keeping accuracy stable and higher than 0.862 for uniform dataset and 0.807 for skew dataset by searching nearest 64 postings, the memory and CPU utilization keep in about 74 GB and 1300% in both uniform and skew datasets.

search thread to 8, the QPS of SPFresh and the IOPS of Disk will no more grows, so in stress test we will set search thread to 8 and then maximize the update throughput, and we find out that when fore-ground thread set to 4 the IOPS will reach the peak during the update.

5.3.2 Experiment Results.

Figure 9 records a daily time series of the search P99.9 latency on both uniform and skew datasets, search/insert throughput, and the IOPS of Work-Load C. We can see that SPFresh reaches the IOPS limitation with stable performance and resource utilization throughout the entire run.

High NVMe SSD IOPS Utilization: As we can see from Figure 9, SPFresh always fully utilizes the NVMe's bandwidth, even exceeding the max *guaranteed* IOPS of Azurelsv3. Thanks to LIRE's lightweight protocol, we can see SPFresh's bottleneck is in the disk IOPS, before reaching the CPU and memory resource limit.

Stable Search and Update Performance: As the data scale grows from 100M to 1B, the search latency is stable, just like that in §5.2. There is some slight increase of P99.9 latency in the beginning when the first split jobs are triggered. In this case, the P99.9 latency increases slightly because of the gradual growth of in-memory index size, which makes the in-memory computation more costly over time.

Stable Accuracy: During the entire stress test, the accuracy of SPFresh remains stable, which is higher than 0.862 for the uniform dataset and 0.807 for the skew dataset by searching the nearest 64 postings.

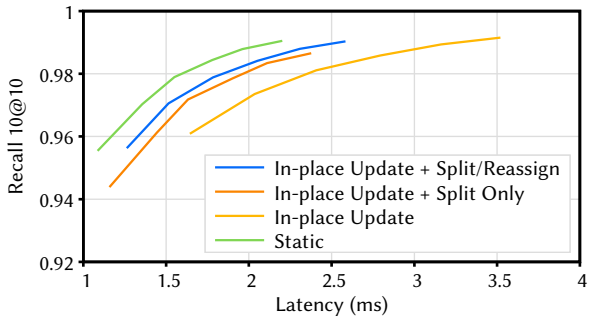

Figure 10. The trade-off on search accuracy and latency with various update techniques under an increasingly skewed data distribution.

5.4 Data Distribution Shifting Micro-benchmark

In this experiment, we replay the experiment in §2.3 to demonstrate that LIRE is required to re-balance the shifting data distribution. We compare four systems in this experiment, where *Static* is our target since it has no updates. For the rest of the three systems, we start with a naive system with in-place update only, i.e., SPANN+, and gradually add sub-components of LIRE into the system.

Experiment Results: Figure 10 shows the recall and latency trade-off result. With a relaxed search latency, the figure shows that recall improves for all four systems. Meanwhile, as the curve moves northwest, the system shows a higher ANNS index quality with a more accurate recall and a lower latency. An in-place update-only solution may have a high

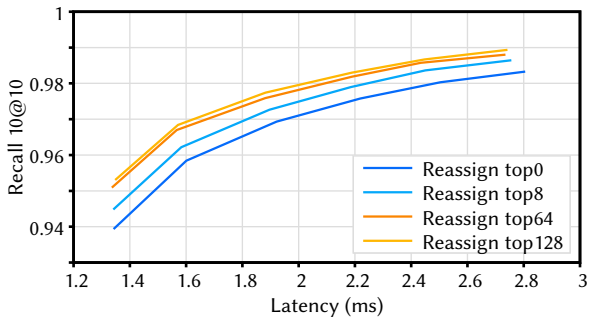

Figure 11. Parameter Study: Reassign Range, top64 (nearest 64 postings) is enough for reassign scanning.

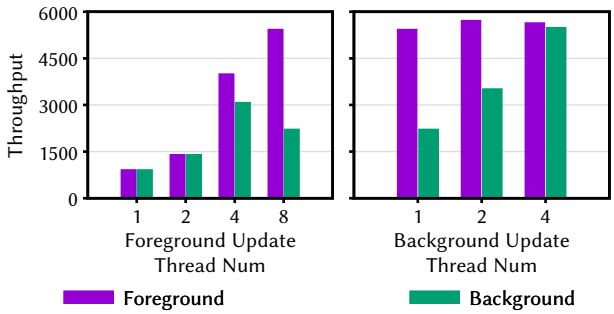

Figure 12. Foreground Scalability (Background Thread Number = 1) and Background Scalability (Foreground Thread Number = 8).

recall but at the expense of high latency. Adding a split component into the in-place update decreases the search latency with the same accuracy. Adding the reassignment component further decreases the search latency. As shown in Figure 10, the performance of SPFresh with in-place update + split/reassign is the closest one to the Static's results, which represent the ideal cases.

5.5 Parameter Study

In this experiment, we investigate the proper parameter configurations for SPFresh to achieve maximum performance. Experiment results show that the Reassignment only requires checking a limited scope of proximate postings for scanning to attain a good index quality. Furthermore, SPFresh demands a minimal increase in computational resources, specifically in terms of threads, to accomplish high throughput while also demonstrating good scalability.

Reassign Range: The first parameter we examine is the reassign range, i.e., the size of the local rebuild range. Reassign range is measured by the number of nearby postings to check for vector reassignment after a new posting list is created. In this experiment, we use the same setting as in §2.3.

In Figure 11, we vary reassign range from the nearest 0, i.e., only process reassign in the split posting, to the nearest 128 postings. As the reassign range increases, accuracy also increases with the same search time budget because more NPA-violating vectors are identified and reassigned. The accuracy increase rate wanes off as the reassign range increases, where there is only a marginal increase from range 64 to 128. Consequently, we chose 64 as SPFresh's default reassign range.

Fore/Back-ground Update Resource Balance: The foreground *In-place Updater* and background *Local Rebuilder* work as a feed-forward pipeline as detailed in §4.2. In this experiment, we examine the proper resource ratio for *In-place Updater* and *Local Rebuilder* to make their processing speed balanced in the pipeline. Specifically, we configure the foreground and background threads and measure the throughput to see when the update resource is balanced.

As the left part of Figure 12 shows, a single-threaded background *Local Rebuilder* can keep up the foreground *In-place Updater* until foreground threads are set to 2. Similarly, as on the right side of Figure 12, an 8-threaded foreground *In-place Updater* needs at least four threads for foreground *In-place Updater* to generate enough requests for the *Local Rebuilder*. Based on the result, SPFresh sets a thread ratio of 2:1 between the foreground *In-place Updater* and the background *Local Rebuilder* balances the feed-forward pipeline.

6 Conclusion

SPFresh supports incremental in-place update for billion-scale vector search. It implements LIRE, a Lightweight Incremental RE-balancing protocol to *split* overly large postings and *reassign* vectors across neighboring postings when necessary. Experiments show that SPFresh can incorporate continuous updates faster with significantly lower resources than existing solutions while maintaining high search recalls by (1) LIRE identifies a minimal set of neighborhood vectors in the large index space for updating to adapt to data distribution shift; (2) the index re-balancing operations and the foreground queries are decoupled, and handled by efficient concurrency control mechanisms, avoiding operation interference. SPFresh's solid single-node performance builds a strong foundation for the future distributed version.

Acknowledgments

We thank all the anonymous reviewers for their insightful feedback, and our shepherd, Nitin Agrawal, for his guidance during the preparation of our camera-ready submission. This work is supported in part by the National Natural Science Foundation of China under Grant No.: 62141216, 62172382 and 61832011, and the University Synergy Innovation Program of Anhui Province under Grant No.: GXXT-2022-045. Cheng Li and Qi Chen are the corresponding authors.